Desarrollo y Despliegue de Soluciones *Big Data*

Guía del proyecto final

1. Introducción

El propósito de este documento es servir como guía integral y transversal para el desarrollo del proyecto de la asignatura, abarcando la totalidad del ciclo de vida de una solución de *big data*. A diferencia de enfoques tradicionales centrados exclusivamente en la algoritmia, este texto estructura el trabajo en fases secuenciales e interdependientes, alineadas con los hitos de seguimiento del curso, para garantizar que la solución final sea robusta, escalable y productiva.

El documento organiza el desarrollo de la solución *big data* en cuatro etapas clave:

1. **Alcance y viabilidad:** formalización de la **necesidad de negocio**, evaluación del **retorno de la inversión (ROI)** y análisis de **riesgos éticos** antes de iniciar el desarrollo técnico, así como la **planificación** del proyecto y la asignación de recursos.
2. **Preparación y gestión de datos:** diseño de ***pipelines* de ingestión**, **validación de esquemas**, **ingeniería de características** (*feature engineering*) y creación de **almacenes de características** (*feature stores*) para asegurar la calidad del dato.
3. **Modelado y experimentación:** entrenamiento de modelos, optimización de **hiperparámetros** y selección de **métricas técnicas** alineadas con los indicadores clave de rendimiento (KPIs) del negocio.
4. **Despliegue y monitorización:** puesta en producción del sistema, estrategias de despliegue y vigilancia continua para detectar la **degradación del modelo** (*data drift* y *concept drift*).

A fin de ilustrar la redacción y el nivel de detalle requerido en cada apartado de la memoria, se utilizará un caso de uso transversal: la **detección de fraude en tarjetas de crédito**.

Este ejemplo servirá de hilo conductor a lo largo de todas las secciones, evidenciando cómo un problema de negocio abstracto se transforma, fase a fase, en un sistema técnico desplegado y monitorizado. Su objetivo es proporcionar una referencia tangible sobre la profundidad de análisis y la estructura argumental esperada en cada hito del proyecto.

2. Alcance y viabilidad

Esta sección establece los cimientos del proyecto. Antes de abordar el desarrollo técnico, es necesario validar que el problema de negocio justifica la inversión en una arquitectura de *big data* y que la organización dispone de los recursos y datos necesarios para ejecutarla con éxito.

2.1. Definición del problema de negocio

En este apartado debéis incluir la definición del **problema de negocio** que ha sido asignado específicamente a vuestro grupo.

Dado que esta asignatura simula un encargo profesional con requisitos estrictos, **vuestra única tarea aquí es copiar textualmente** el enunciado facilitado por el profesorado. Es obligatorio que **no modifiquéis ninguna cifra, métrica o descripción** del escenario (volúmenes de datos, costes unitarios o tasas de error), ya que este texto constituye la *verdad fundamental* del proyecto y garantiza la coherencia con los conjuntos de datos que recibiréis en la fase de ingeniería.

A modo de referencia ilustrativa, a continuación se presenta la redacción correspondiente al caso de uso transversal de la asignatura (**detección de fraude**). Como podéis ver en el ejemplo, el texto describe el problema y cuantifica su impacto económico y operativo con métricas precisas, siguiendo el formato exacto que debéis replicar en vuestra memoria.

Caso de uso: fraude en tarjetas de crédito

La entidad financiera opera en un escenario de alta actividad comercial, gestionando una cartera con un volumen medio de aproximadamente **1.000.000 de operaciones mensuales** y un *ticket* promedio de **50 euros**. En este contexto, la organización se enfrenta a una tasa de intentos de fraude del **1 %** (10.000 ataques mensuales), una cifra en tendencia alcista que los procedimientos actuales, basados en reglas manuales rígidas, son incapaces de contener.

Esta limitación operativa genera una pérdida financiera crítica por dos vías. Por un lado, la ineficacia de los controles actuales permite que se materialicen **4.000 fraudes reales** cada mes (una tasa de no-detección del 40 %), lo que obliga al banco a asumir directamente el coste de lo sustraído, ascendiendo a **200.000 euros mensuales** en pérdidas directas.

Por otro lado, la agresividad de las reglas deteriora la experiencia del usuario, bloqueando erróneamente el **5 % de las compras legítimas** (50.000 operaciones denegadas incorrectamente). Considerando el impacto económico de rechazar a un cliente válido (estimado en **25 euros** por costes de gestión y riesgo de abandono), esta ineficiencia añade **1.250.000 euros** a la factura mensual. En conjunto, el sistema actual provoca un agujero financiero total de **1.450.000 euros al mes**, haciendo insostenible el mantenimiento del «status quo».

2.2. Planteamiento y selección de la solución técnica

Una vez diagnosticado el problema, el siguiente paso es **evaluar las distintas alternativas** posibles para resolverlo. En ingeniería de datos, debéis recordar siempre que la **solución más compleja no siempre es la mejor**; a menudo, una solución sencilla bien implementada aporta más valor que una arquitectura distribuida innecesaria.

Por tanto, en este apartado no basta con elegir una tecnología; debéis **plantear y analizar críticamente** al menos tres escenarios: una **solución heurística** (basada en reglas manuales), una solución de **aprendizaje tradicional** (para pequeños volúmenes de datos) y, finalmente, la **solución *big data***.

Vuestra tarea consiste en **descartar razonadamente** las dos primeras opciones y **justificar la elección** de la arquitectura *big data*. Esta justificación debe basarse en las características intrínsecas del problema, utilizando las «Vs» del *big data* como argumento: **volumen** masivo de históricos, **velocidad** de respuesta en tiempo real o la **complejidad** no lineal de los patrones a detectar.

Advertencia sobre la coherencia técnica: El siguiente cuadro ilustra un caso de *fraude en tiempo real*, por lo que el argumento principal para descartar el servidor tradicional es la **velocidad** (latencia de respuesta crítica).

Dado que vuestros proyectos pueden ser de naturaleza diferente, vuestra justificación debe adaptarse. No copiéis las restricciones de «milisegundos» si no aplican a vuestro caso; en su lugar, fundamentad la necesidad de una arquitectura distribuida basándoos en las otras **«Vs» del *big data***: quizás vuestro cuello de botella no sea la velocidad, sino el **volumen** (imposibilidad de procesar todo el histórico en la memoria de un solo servidor) o la **variedad** (complejidad dimensional de los datos que requiere algoritmos no lineales).

Caso de uso: fraude en tarjetas de crédito

Para determinar la arquitectura óptima, el equipo técnico sometió a evaluación tres escenarios posibles. En primera instancia, se descartó la optimización del **sistema heurístico actual**, dado que la gestión manual de reglas ha demostrado ser insostenible operativamente e incapaz de frenar el fraude sin disparar la tasa de falsos positivos (**bloqueo erróneo de clientes legítimos**).

Posteriormente, se analizó la viabilidad de una solución de **aprendizaje automático tradicional** basada en un servidor monolítico (escalado vertical). Si bien esta opción simplificaría el desarrollo, presenta riesgos estructurales inaceptables. Por un lado, la necesidad de entrenar modelos complejos utilizando **ventanas históricas plurianuales** (necesidad de cargar en memoria años de datos pasados para capturar la estacionalidad) saturaría un servidor convencional, impidiendo la iteración rápida. Por otro lado, un sistema centralizado carece de la elasticidad necesaria para absorber los **picos de carga estacionales** (como en campañas de *Black Friday*) sin degradar la latencia, lo que pondría en riesgo la operatividad.

Por consiguiente, la elección final recae sobre una **arquitectura *big data* distribuida**. Esta decisión es la única que garantiza el cumplimiento de las variables críticas: capacidad de cómputo paralelo para digerir el **volumen** histórico requerido, escalabilidad horizontal para mantener la **velocidad** ante picos de demanda masiva, y flexibilidad para adaptar los modelos ante la **variabilidad** de los patrones de fraude.

2.3. Evaluación de la viabilidad y valor

Una vez seleccionada la arquitectura, es obligatorio someter la propuesta a un riguroso análisis de factibilidad. En este punto, debéis demostrar que el proyecto trasciende la teoría para convertirse en una iniciativa ejecutable y rentable. El análisis se estructura en tres dimensiones críticas: la **viabilidad técnica**, centrada en la idoneidad de los datos (¿existe histórico suficiente y accesible?); la **viabilidad económica**, donde debéis estimar el **ROI** para probar que el valor generado supera los costes de infraestructura; y, finalmente, la **viabilidad ética y legal**.

Este último pilar constituye un requisito indispensable. Debéis realizar una auditoría preventiva para identificar riesgos de **privacidad** (cumplimiento normativo) y posibles **sesgos algorítmicos** (*fairness*). Un modelo que genere beneficios económicos pero discrimine a un colectivo protegido o exponga datos sensibles carece de viabilidad y debe ser replanteado desde su diseño.

Nota sobre la base de cálculo: Para completar la **viabilidad técnica**, deberéis esperar a recibir el conjunto de datos asignado, momento en el que podréis especificar los años de histórico y el volumen exacto de transacciones disponibles.

Para la **viabilidad económica**, las métricas de mejora deben basarse estrictamente en el problema de negocio definido en el apartado anterior. Es fundamental que seáis **conservadores** en vuestras estimaciones (no prometáis eliminar el 100 % del error). Respecto a los costes de infraestructura, podéis utilizar la estimación proporcionada en el siguiente ejemplo o realizar vuestro propio cálculo utilizando la calculadora de precios de Databricks. Finalmente, respecto a la auditoría ética, dado que los algoritmos de equidad (*fairness*) se verán al final de la asignatura, se recomienda que **consultéis con un modelo de lenguaje** cuál es la métrica más indicada para vuestro caso de uso específico y la citéis en este apartado.

Caso de uso: fraude en tarjetas de crédito

El análisis de factibilidad ratifica la solidez del proyecto en sus tres dimensiones críticas. En el plano de la **viabilidad técnica**, la organización parte de una posición ventajosa al disponer de un **repositorio histórico saneado** que abarca los últimos **5 años de operativa**. Este activo de datos, que suma aproximadamente **60 millones de transacciones etiquetadas**, garantiza el volumen necesario para entrenar modelos profundos sin riesgo de sobreajuste.

Desde la perspectiva de la **viabilidad económica**, la proyección es contundente. Partimos de las pérdidas actuales de **1.450.000 euros mensuales** (desglosadas en 200.000 euros por fraude directo y 1.250.000 euros por fricción operativa o bloqueos indebidos). Se ha modelado un escenario de mejora **conservador**. En primer lugar, una reducción del 20 % en falsos positivos (fricción):

$$1{,}250{,}000 \text{ euros} \times 0,20 = \mathbf{250.000\ euros}.$$

Por otro lado, una mejora del 10 % en detección de fraude:

$$200{,}000 \text{ euros} \times 0,10 = \mathbf{20.000\ euros}.$$

Esto genera un **ahorro total de 270.000 euros mensuales**. Para calcular el coste, sumamos el equipo de ingeniería (dos especialistas a **5.000 euros al mes** cada uno) y la infraestructura en la nube. Según la calculadora oficial, un clúster de dos instancias `m5.2xlarge` (necesarias para procesar el volumen de datos en memoria) operando 24 horas supone un coste aproximado de **1.100 euros al mes**.

Con un coste total de 11.100 euros y un beneficio de 270.000 euros, el proyecto arroja un **ROI mensual del 2.332 %**, permitiendo amortizar la inversión tecnológica en apenas **1,3 días de operación**.

Finalmente, para asegurar la **viabilidad ética**, se han establecido líneas rojas. Todos los identificadores personales serán sometidos a **seudonimización**. Asimismo, tras consultar la literatura especializada, se ha determinado que la métrica de equidad prioritaria será la **igualdad de oportunidades** (*equal opportunity*), para asegurar que la tasa de detección de fraude sea consistente independientemente de la edad del cliente.

2.4. Planificación y recursos

Para cerrar la fase de definición, es imprescindible trazar la hoja de ruta operativa del proyecto. Un buen modelo predictivo es inútil si no se puede medir su impacto o si no se define claramente cómo se integrará en el entorno productivo. Por ello, en este apartado debéis detallar los pilares fundamentales de la ejecución: las **métricas de éxito**, diferenciando claramente entre los **KPIs de negocio** y las **métricas técnicas**; el **equipo y las fuentes de datos**; y el ***stack* tecnológico** seleccionado. Adicionalmente, debéis describir la integración de herramientas y, para garantizar la ejecución ordenada, incluir una **planificación temporal** basada estrictamente en los hitos oficiales.

Guía de adaptación: Para la redacción de este apartado, es fundamental que distingáis entre los elementos variables y las restricciones fijas de la asignatura. El primer párrafo del ejemplo (métricas de éxito) es **estrictamente dependiente de vuestro problema** y debe recalcularse: debéis traducir la promesa económica del apartado anterior (ROI) a las métricas técnicas exactas que gobiernen vuestro modelo. En escenarios de

clasificación, definid las tasas de acierto o los umbrales de confusión necesarios para materializar el ahorro estimado; mientras que si vuestro problema es de **regresión**, deberéis establecer el margen de error máximo tolerable para que la predicción siga aportando valor (ya sea en términos absolutos o porcentuales).

Por el contrario, el resto de la hoja de ruta operativa actúa como un estándar común. Podéis mantener la descripción del **equipo** (dos especialistas), la **arquitectura de datos dual** (tablas de contexto y eventos), el ***stack* tecnológico** y los **hitos temporales**, ya que estos recursos y plazos aplican por igual a todos los grupos de trabajo.

Caso de uso: fraude en tarjetas de crédito

La hoja de ruta operativa traduce las promesas económicas anteriores en umbrales técnicos precisos que actuarán como garantes del proyecto. Para materializar el ROI esperado, el modelo debe cumplir dos objetivos matemáticos estrictos derivados de la situación actual.

En primer lugar, respecto a la seguridad, partimos de una tasa de detección base del 60 % (4.000 fraudes detectados sobre 10.000). Para recuperar los 20.000 euros mensuales prometidos (mejora del 10 %), el nuevo modelo está obligado a elevar dicha tasa (*recall*) hasta el $60\,\% \times (1 + 0,10) = \mathbf{66}\,\%$.

Simultáneamente, para reducir los costes de fricción, debemos actuar sobre la tasa de bloqueos erróneos (base actual del 5 %). Dado que nos hemos comprometido a reducir este error en un 20 %, el objetivo técnico innegociable será mantener los falsos positivos por debajo del $5\,\% \times (1 - 0,20) = \mathbf{4}\,\%$.

Solo cumpliendo ambas cifras se sostienen las cuentas presentadas. La ejecución técnica para lograrlo recae sobre un equipo multidisciplinar de **dos especialistas** que unifican los roles de **ingeniería de datos** y **ciencia de datos**. Este enfoque *end-to-end* es necesario para orquestar el ciclo completo: desde la gestión del **repositorio de datos híbrido** (que combina tablas de contexto histórico con flujos de eventos en **ficheros `.json` particionados por año y mes**) hasta la **puesta en producción**, abarcando la experimentación iterativa, el **enriquecimiento de características** (*feature lookup*), el entrenamiento de los algoritmos y su despliegue final en la infraestructura *cloud*.

En cuanto a la arquitectura tecnológica, el flujo se centraliza en la plataforma **`Databricks`**. El procesamiento masivo y el entrenamiento matemático se delegarán en la librería **`Spark MLlib`**, aprovechando su capacidad de paralelización. La gobernanza del ciclo de vida quedará asegurada mediante **`MLflow`**, herramienta encargada del registro de experimentos y trazabilidad. Finalmente, la planificación temporal se estructura en cuatro hitos críticos: el cierre del alcance para el **27 de febrero de 2026**, la finalización de la ingeniería de datos el **27 de marzo de 2026**, la entrega del modelo validado el **24 de abril de 2026** y el despliegue final con monitorización para el **1 de mayo de 2026**.

Consideraciones finales

El desarrollo de este apartado debe seguir el **estilo narrativo y la profundidad de análisis** mostrados en los bloques de ejemplo (caso de uso de fraude). No obstante, estos ejemplos constituyen una referencia base: tenéis total libertad para **ampliar el alcance**, profundizar en la justificación técnica o incorporar detalles adicionales si la naturaleza de vuestro problema de negocio lo requiere.

En cuanto al formato, la extensión total recomendada para esta sección (cubriendo definición, solución, viabilidad y planificación) oscila entre las **2 y 4 páginas**. Se valorará positivamente la capacidad de síntesis y el uso estratégico de **elementos visuales** (diagramas de arquitectura, tablas resumen o cronogramas) para enriquecer el texto sin caer en una redacción excesivamente extensa o redundante.

3. Preparación y gestión de datos

3.1. Configuración del entorno de trabajo y control de versiones

El primer paso para el desarrollo de cualquier proyecto de ingeniería de datos a nivel profesional es establecer un entorno de trabajo colaborativo y un control de versiones robusto.

En primer lugar, los integrantes del equipo deben registrarse en **Databricks** y configurar un espacio de trabajo (*workspace*) compartido. Para colaborar eficazmente y evitar bloqueos de permisos, es necesario que ambos miembros se concedan permisos de administración mutuos sobre el entorno. Para ello, desde la interfaz principal de **Databricks**, se debe navegar a través de:

`Settings` → `Identity and access` → `Users` → `Add user` → `Add new` e introducir el correo electrónico del compañero. Asimismo, se deberá conceder acceso al equipo docente para facilitar las labores de tutorización. Una vez añadidos los usuarios, se les debe incluir en el grupo de administradores navegando a: `Groups` → `admins` → `Members` → `Add members`.

Tras asegurar el acceso, el equipo deberá crear un repositorio en **GitHub** destinado a almacenar y versionar el proyecto. El uso de **Git** es una buena práctica indispensable en la industria. Se valorará positivamente no solo la creación del repositorio, sino su correcta estructuración, modularidad y documentación a través de un fichero **README.md**.

Una vez creado el repositorio, este se debe vincular al espacio de trabajo de **Databricks**. Para importar el proyecto, se debe acceder a `Workspace` → `Create` → `Git folder`, introducir la dirección del repositorio, seleccionar el proveedor (**GitHub**) junto con el nombre de la carpeta, y pulsar finalmente en `Create Git Folder`.

3.2. Infraestructura de datos y gobernanza

Antes de escribir una sola línea de código para la ingesta, es obligatorio establecer los cimientos donde vivirán nuestros datos. En un entorno *big data* moderno, no se trabaja con archivos sueltos dispersos, sino con **objetos gobernados**.

Para este proyecto, utilizamos **Unity Catalog**, el cual nos permite estructurar los datos jerárquicamente. Esta estructura garantiza la seguridad, el linaje del dato y la organización lógica del proyecto. La jerarquía de gobernanza consta de tres niveles principales:

- ***Catalog* (Catálogo)**: Es el contenedor de nivel superior y representa la unidad de gobernanza más alta. Haciendo una analogía, actúa como el disco duro físico o la unidad de red corporativa de la organización.
- ***Schema* (Esquema)**: Es la agrupación lógica de objetos (tablas, volúmenes, modelos, etc.) que pertenecen a un dominio de negocio o proyecto específico. Siguiendo con la analogía, representaría la carpeta raíz de un departamento o proyecto.
- ***Data objects* (Objetos de datos)**: Es el nivel inferior donde residen físicamente los activos. Pueden ser principalmente de dos tipos: **tablas** (datos estructurados relacionales en filas y columnas) o **volúmenes** (datos no estructurados o semiestructurados, como archivos planos, imágenes o binarios).

3.2.1. Creación de la *landing zone* y carga de datos

Llevando la teoría anterior a nuestro caso práctico, el primer paso es definir nuestra estructura. Por las restricciones propias del entorno educativo, utilizaremos el catálogo predeterminado de la plataforma llamado **workspace**, que actuará como nuestro directorio compartido principal.

Dentro de este, necesitamos agrupar nuestro proyecto. Para ello, debéis dirigiros al apartado de `Catalog`, seleccionar el catálogo `workspace`, pulsar `Create schema`. Asignadle el nombre que consideréis más apropiado para vuestro dominio (por ejemplo, **`credit_card_fraud`** en el caso del proyecto de detección de fraude) y confirmad pulsando `Create`. Todo lo que se procese en vuestro proyecto vivirá contenido de manera aislada dentro de este esquema.

Una vez creado el esquema, necesitamos configurar nuestra zona de ingesta cruda. Uno de los conceptos más importantes en la ingeniería de datos moderna es la separación física y lógica entre el almacenamiento de archivos y las tablas gestionadas. Para lograr esto, dentro de nuestro esquema crearemos un **volumen gestionado** llamado **`landing_zone`**. Para ello, con el esquema recién creado seleccionado, pulsad `Create` → `Volume`. Introducid **`landing_zone`** como nombre y confirmad pulsando `Create`.

A diferencia de una tabla tradicional, un volumen en **`Databricks`** es un punto de montaje para almacenamiento de objetos en la nube. Aquí es donde los sistemas externos depositarán los archivos originales (**`.csv`**, **`.json`**, etc.) antes de ser procesados.

3.2.2. Estrategia de organización de datos

Para garantizar la eficiencia en la lectura y el escalado del sistema, no podemos tratar el volumen recién creado como un «cajón desastre». Adoptaremos una estrategia técnica estándar en la industria conocida como ***hive partitioning***.

Esta metodología consiste en organizar los archivos en una estructura de directorios anidados basada en los valores de una o más columnas (normalmente fechas). Su gran ventaja es que permite a los motores de procesamiento como **`Spark`** realizar *partition pruning* (poda de particiones), es decir, leer únicamente las carpetas estrictamente necesarias para resolver una consulta y omitir el resto del histórico.

A modo de referencia, tomando como ejemplo el proyecto de detección de fraude, la estructura inicial del volumen **`landing_zone`** constaría de los siguientes directorios (debéis adaptar esta lógica a la naturaleza de vuestros propios datos):

- **`context` (Datos contextuales)**: Almacena los datos maestros o dimensionales (por ejemplo, el histórico de clientes o productos). Estos datos sirven para enriquecer las transacciones. Residen en archivos consolidados (**`.csv`**) que ya incluyen marcas de tiempo internas para gestionar su vigencia. Por su naturaleza estática o de actualización lenta, no suele aplicarse particionamiento físico en este nivel.
- **`events` (Eventos transaccionales)**: Almacena el flujo continuo e incesante de eventos en archivos **`.json`**, organizado a su vez en dos subdirectorios diferenciados:
  - **`events/transactions`**: contiene los registros de cada operación en el momento en que se produce.
  - **`events/labels`**: contiene las etiquetas de fraude asociadas a cada transacción. La separación física de ambos subdirectorios es deliberada: en un entorno real, la confirmación de si una operación es fraudulenta puede tardar días o semanas en llegar (*delayed feedback*), por lo que mantenerlas desacopladas permite ingerir y procesar cada flujo de forma independiente.

  Dado que ambos subdirectorios crecen de manera indefinida, se aplica en los dos un particionamiento temporal con una estructura por año y mes, de forma que cada carpeta hoja contiene únicamente los archivos correspondientes a ese periodo.
- **`source_buffer` (Búfer de origen)**: Esta carpeta contiene los «datos del futuro» correspondientes a los eventos más recientes. Se utilizará en fases posteriores de la asignatura para simular la llegada de nuevas transacciones en un entorno de *streaming* en tiempo real.

Antes de proceder con la subida a **`Databricks`**, es requisito indispensable haber ejecutado en vuestro entorno local el *script* de generación de datos correspondiente a vuestro proyecto. Dicho *script* se encargará de crear la estructura de carpetas poblada de datos que deberéis replicar en la nube.

Para efectuar la carga, acceded al volumen **landing_zone**, haced clic en `Upload to this volume`, seleccionad y subid las carpetas generadas por vuestro *script* (en nuestro ejemplo: **context**, **events** y **source_buffer**), y pulsad en `Upload`.

Nota importante para la memoria del proyecto: En este punto del documento, se recomienda detallar la estructura de directorios y ficheros resultante tras la generación de vuestros datos. Deberéis documentar las carpetas que conforman vuestra *landing zone*, el formato de los archivos que contienen (por ejemplo, **.csv**, **.json**) y explicar brevemente la estrategia de particionado adoptada, indicando las columnas utilizadas como clave de partición y el esquema de directorios resultante.

3.3. Automatización y orquestación

Una vez preparado el entorno y la zona de aterrizaje de los datos, comenzaremos a construir la arquitectura para nuestro proyecto, para la cual utilizaremos el paradigma ya conocido: la **arquitectura *Medallion*** (capas bronce, plata y oro).

Hasta este momento en la asignatura, habéis escrito *scripts* de manera aislada y os habéis encargado de ejecutarlos manualmente en el orden correspondiente. En un escenario del mundo real, este enfoque es inviable. Se necesita un proceso automatizado que detecte las dependencias entre los distintos procesos, gestione la infraestructura subyacente y sincronice los datos de manera continua. Para ello, utilizaremos el orquestador nativo ***Delta Live Tables (DLTs)***.

Para instanciar este orquestador, debemos ir a `Jobs & Pipelines` → `ETL pipeline`. Una vez dentro, le asignaremos un nombre descriptivo (por ejemplo, **Credit Card Fraud Medallion Pipeline** adaptado a vuestro caso). A continuación, marcaremos la opción `Set up a source-controlled project` → `Create new project` y vincularemos la carpeta del repositorio **Git** que importasteis anteriormente.

Tras esta configuración, se generará automáticamente una estructura de directorios profesionalizada, que contiene los siguientes elementos clave:

- **.vscode**: Configuraciones específicas para el entorno de desarrollo visual, asegurando que se mantengan los estándares de formato del código.
- **resources**: Directorio destinado a la infraestructura como código. Contiene las definiciones (en formato **.yml**) que indican a **Databricks** qué clústeres y recursos computacionales instanciar para ejecutar el *pipeline*.
- **src**: El directorio principal que alberga el código fuente (**Python**) de nuestro proyecto.
- **.gitignore**: Archivo estándar de **Git** que especifica qué archivos locales no deben subirse al repositorio.
- **databricks.yml** y **pyproject.toml**: Archivos de configuración general del proyecto y gestión de dependencias.
- **README.md**: Documento base para explicar el propósito y la arquitectura del repositorio.

Si exploramos el interior de la carpeta **src** y entramos en el directorio con el nombre de nuestro *pipeline*, encontraremos dos carpetas fundamentales generadas automáticamente:

- **explorations**: Espacio reservado para los *notebooks* de análisis exploratorio de datos, prototipado rápido y visualizaciones que no forman parte del flujo de producción. **En el contexto de este proyecto, la fase de exploración no es un entregable obligatorio**; no obstante, se valorará positivamente que el equipo documente en este directorio un análisis exploratorio previo al modelado, ya que refleja rigor metodológico y facilita la justificación de las decisiones de ingeniería tomadas en fases posteriores.

- **`transformations`**: El núcleo del motor DLT. En este directorio alojaremos nuestros *scripts* de **`Python`** encargados de materializar las diferentes capas de la arquitectura. Cualquier *script* depositado aquí será analizado por DLT para inferir automáticamente el grafo de dependencias y ejecutarlo.

3.4. Ingesta de datos: capa bronce

El primer paso algorítmico que debemos llevar a cabo es la ingesta hacia la capa bronce (*raw layer*). El objetivo de esta capa es almacenar una copia exacta e inmutable de los archivos originales, pero convertidos al formato columnar de alto rendimiento **`Delta Lake`**.

Para ello, deberéis crear un *script* en **`transformations`** (por ejemplo, **`01_bronze_ingestion.py`**). Dentro de este, notaréis una gran diferencia metodológica: el proceso de creación de tablas ya no se hace de manera **imperativa** (indicando el «cómo» paso a paso mediante **`.write.save`**), sino de manera **declarativa** (indicando el «qué» queremos obtener). Esto se logra mediante los decoradores de **`pyspark.pipelines`**, la librería de código abierto (importada convencionalmente como **`import pyspark.pipelines as dp`**) que permite definir *pipelines* de datos de manera declarativa. El decorador principal que utilizaréis en esta fase es **`@dp.table`**, y el orquestador se encargará por debajo de gestionar transacciones, fallos e infraestructura adaptando su comportamiento automáticamente tanto a lecturas *batch* como en *streaming*.

Otro aspecto vital a tener en cuenta en el código es el método de lectura de datos, que variará según la naturaleza de la tabla:

- **Modo *batch* (`spark.read`)**: Se utiliza para datos maestros o dimensionales (como la tabla de clientes o catálogo de productos). Al ser un estado o «foto fija», se realiza una lectura tradicional que sobrescribirá los datos en cada ejecución.
- **Modo *streaming* (`spark.readStream` y formato `cloudFiles`)**: Se utiliza para las transacciones o eventos. Esta instrucción invoca el **`Auto Loader`** de **`Databricks`**, indicando que estamos ante un flujo de datos incremental. Esto permite monitorizar la carpeta del volumen de forma continua y crear *checkpoints* (puntos de control).

> Los *checkpoints* actúan como la «memoria» de la ingesta: registran exactamente en qué archivo se detuvo el procesamiento en la última ejecución. Gracias a ello, si llegan archivos nuevos, el sistema procesará únicamente la diferencia respecto al estado anterior, garantizando eficiencia y eliminando el riesgo de datos duplicados.

3.4.1. Ejecución y verificación

Para ejecutar el código, regresaremos a la pestaña `Jobs & Pipelines`, seleccionaremos nuestro *pipeline* y pulsaremos `Start`. Por defecto, la ejecución será incremental. Si en algún momento, durante la fase de desarrollo, necesitáis reprocesar todo el histórico omitiendo los *checkpoints*, podéis desplegar las opciones y seleccionar `Run pipeline with full refresh all`.

Una vez finalizada la ingesta con éxito, si nos dirigimos a la pestaña `Catalog`, veremos que, bajo el esquema que creamos previamente, se han generado las tablas correspondientes a la capa bronce (por ejemplo, **`bronze_customers`**, **`bronze_labels`** y **`bronze_transactions`** en el caso de referencia).

3.4.2. Metadatos de auditoría

Al explorar estas tablas en la pestaña `Sample Data`, observaréis que, además de las columnas de negocio, se han generado columnas técnicas automáticas. Nótese la importancia crítica de metadatos como:

- **`_rescued_data`**: Permite capturar discrepancias o fallos de esquema de manera segura sin detener la ingesta.
- **`ingestion_timestamp`**: Registra el instante temporal exacto en el que el dato entró a nuestro ecosistema analítico, clave para la trazabilidad y la depuración temporal.
- **`source_file`**: Almacena la ruta absoluta del archivo origen del cual procede esa fila en concreto, fundamental para auditorías.

Nota importante para la memoria del proyecto: En este punto del documento, deberéis detallar exhaustivamente la información de las tablas de la capa bronce que habéis generado para vuestro proyecto específico. Esto incluye: enumerar las tablas creadas, mostrar el recuento total de instancias (filas) de cada una tras la carga histórica, y documentar el esquema de columnas (nombre, tipo de dato y su significado de negocio). Comprender íntimamente estas variables crudas ahora es imprescindible para pensar, de cara a fases posteriores, qué características de aprendizaje automático podríais derivar.

3.5. Refinamiento y calidad de datos: capa plata

Una vez completada la ingesta cruda en la capa bronce, el siguiente paso natural en la arquitectura *Medallion* es refinar esos datos. En la capa plata (*silver layer*), el objetivo es limpiar, validar, tipar y cruzar la información proveniente de múltiples orígenes para obtener un modelo de datos unificado y confiable, listo para el análisis y el modelado.

3.5.1. Definición de reglas de calidad

En un entorno de producción moderno, la calidad del dato no se valida mediante condicionales **`if-else`** esparcidos por el código, sino mediante **reglas declarativas** (conocidas en DLT como *expectations*).

Para mantener el código limpio, escalable y modular, es una buena práctica crear un subdirectorio llamado **`rules`** dentro del código de vuestro *pipeline*. En su interior, crearéis un *script* de **`Python`** por cada tabla lógica a evaluar (por ejemplo, en el caso de referencia, **`customers.py`**, **`labels.py`** y **`transactions.py`**).

En lugar de incrustar las reglas en duro dentro del flujo de transformación, estos archivos modulares contendrán funciones que devuelven diccionarios con las restricciones. Cada regla debe estar formada por un nombre, una restricción en sintaxis **`SQL`** (por ejemplo, validar que un **`id`** no sea nulo o que un importe sea mayor a cero) y, muy importante, una entrada llamada **`tag`**.

El **`tag`** (etiqueta) es crucial, ya que permite identificar a qué tabla o dominio pertenece cada regla, facilitando su filtrado e importación dinámica en el script principal. Se recomienda además crear un archivo **`__init__.py`** en este directorio para actuar como punto de entrada unificado y facilitar la importación de todas las reglas simultáneamente.

3.5.2. Transformación y cuarentena de datos

Una vez definidas las reglas, el *script* principal de esta fase (por ejemplo, **`02_silver_transformation.py`**) se encargará de aplicarlas.

Un concepto arquitectónico fundamental que debéis implementar aquí es el de **cuarentena** o *dead letter queue (DLQ)*. En procesos críticos, la gestión de registros inválidos no es trivial: DLT ofrece varias estrategias nativas ante el incumplimiento de reglas de calidad, pero ninguna de ellas es adecuada en un entorno de producción. Por un lado, **`@dp.expect_or_drop`** elimina silenciosamente los registros que no superan la validación, lo que implica una pérdida de datos irrecuperable sin dejar rastro auditable. Por otro, **`@dp.expect_or_fail`** detiene completamente el *pipeline* ante el primer registro anómalo, lo que resulta inviable en flujos continuos donde la presencia de datos sucios es esperable. En su lugar, la estrategia correcta consiste en:

1. Crear la tabla física de cuarentena de forma anticipada mediante **`dp.create_streaming_table`**, que actuará como DLQ y recibirá los registros anómalos.
2. Definir una tabla temporal (**`temporary = True`**) decorada con **`@dp.table`** y **`@dp.expect_all`**, que lee los datos crudos de la capa bronce (añadiendo la opción **`skipChangeCommits`** para ignorar operaciones de sobreescritura en la fuente sin interrumpir el *pipeline*) y adjunta a cada registro un *flag* booleano **`is_quarantined`**, construido dinámicamente negando la combinación de todas las reglas de calidad definidas. Esta tabla actúa como *hub* de enrutamiento: evalúa el flujo completo en un único paso sin persistir estado intermedio en disco.
3. Bifurcar el flujo en dos caminos independientes a partir del *flag*:
   - Los registros **inválidos** (**`is_quarantined = true`**) se escriben de forma incremental en la tabla de cuarentena mediante un **`@dp.append_flow`**, descartando el *flag* antes de la escritura.
   - Los registros **válidos** (**`is_quarantined = false`**) se exponen a través de una **`@dp.view`**, que actúa como fuente limpia para los procesos posteriores sin materializar datos adicionales en disco.
4. El flujo válido continúa desde la vista hacia el proceso ***AUTO Change Data Capture (CDC)***, que se encarga de gestionar el histórico de cambios en la capa plata.

Esta arquitectura permite al equipo de gobernanza del dato auditar los errores «*a posteriori*» sin detener el flujo principal de ingesta.

3.5.3. Gestión de históricos

Los atributos de las entidades de negocio cambian con el tiempo (un cliente puede cambiar de nivel de ingresos, o un producto puede cambiar de categoría). Si simplemente sobrescribimos su información en la base de datos, perderíamos el contexto histórico, lo cual es fatal para el aprendizaje automático, ya que los modelos necesitan conocer el estado exacto de una entidad en el momento en que ocurrió un evento (*point-in-time correctness*).

Para resolver esto, aplicamos el concepto de ***Slowly Change Dimension (SCD)*** **Tipo 2**. DLT facilita esto enormemente mediante la funcionalidad de AUTO CDC. Usando la directiva **`dp.create_auto_cdc_flow`**, el motor compara automáticamente la información entrante con la existente basándose en una clave primaria (por ejemplo, **`customer_id`**) y una columna temporal (por ejemplo, **`customer_updated_at`**).

Internamente, el sistema genera y gestiona las columnas técnicas **`__START_AT`** y **`__END_AT`**, manteniendo un registro histórico perfecto y automático de la validez temporal de cada versión del registro. Cabe destacar que, al configurar el flujo CDC, es importante excluir explícitamente mediante **`except_column_list`** las columnas técnicas de auditoría heredadas de la capa bronce, como **`ingestion_timestamp`** y **`source_file`**. De no hacerlo, el motor las interpretaría como atributos de negocio y generaría nuevas versiones históricas del registro ante cualquier cambio en estos metadatos, introduciendo ruido espurio en el histórico.

3.5.4. Enriquecimiento: cruce *stream-stream* y *watermarks*

El último paso de la capa plata suele ser cruzar datos maestros o eventos relacionados para crear una tabla de hechos unificada. Cuando cruzamos dos flujos de datos continuos (*stream-stream join*), nos enfrentamos a un reto técnico considerable: la **llegada tardía de datos** o el *feedback* retardado.

Por ejemplo, una transacción ocurre hoy, pero la confirmación final de si fue un fraude (etiqueta) puede tardar semanas en generarse. Para cruzar estos dos flujos, el motor (**`Spark`**) necesita mantener las transacciones en su memoria de estado esperando a que llegue su correspondencia. Para evitar un error de falta de memoria, se utiliza la técnica de ***watermarking*** (marcas de agua).

Al aplicar la función **`.withWatermark`**, establecemos un límite máximo de espera. Si un registro lleva más tiempo en memoria que el umbral definido por la marca de agua sin encontrar correspondencia, el sistema asume que la información cruzada ya no llegará. En ese momento, finaliza el intento de cruce (dejando los valores cruzados como nulos, en el caso de un **`LEFT JOIN`**), escribe el resultado definitivo en la tabla final consolidada y, crucialmente, libera la memoria de estado de forma segura. La elección del umbral no debe ser arbitraria: se recomienda estimarlo empíricamente a partir de los datos históricos, calculando el retardo máximo observable entre el **`timestamp`** de la transacción y el **`timestamp`** de su etiqueta correspondiente. Un umbral demasiado corto provocará que registros válidos sean descartados prematuramente; uno demasiado largo incrementará innecesariamente el consumo de memoria de estado.

3.5.5. Ejecución y verificación de la capa plata

Al ejecutar de nuevo el *pipeline* desde la interfaz, observaréis cómo el grafo acíclico dirigido (DAG) se amplía automáticamente, calculando las nuevas dependencias sin que hayáis tenido que orquestar el orden de ejecución a mano.

Si nos dirigimos al esquema correspondiente en `Catalog`, veremos materializadas las tablas limpias, los históricos, la tabla unificada de eventos enriquecidos y las tablas de cuarentena.

Nota importante para la memoria del proyecto: En este apartado de vuestra documentación, debéis explicar detalladamente la lógica de refinamiento aplicada a vuestro proyecto. Se os pedirá:

- Listar y justificar analíticamente las reglas de negocio (*expectations*) que habéis diseñado para validar cada tabla.
- Documentar las tablas resultantes de la capa plata, incluyendo un recuento final de instancias válidas.
- Comprobar si existen registros en las tablas de cuarentena. En caso afirmativo, debéis analizar la naturaleza de estos datos anómalos y reflexionar sobre por qué no cumplieron las reglas impuestas.

3.6. Preparación para aprendizaje automático: capa oro

La última fase en la ingeniería de datos es la construcción de la capa oro. Su objetivo es modelar los datos refinados de la capa plata para crear las tablas de características y la tabla base de eventos que consumirán directamente los modelos de aprendizaje automático (tanto en entrenamiento como en inferencia).

Una de las mejores prácticas en el diseño de arquitecturas es la **modularización**. No debemos crear una única y gigantesca tabla plana con toda la información. En su lugar, es recomendable dividir la lógica en diferentes *scripts* y tablas especializadas según su naturaleza temporal y su patrón de acceso.

A modo de referencia, en un proyecto como el de detección de fraude, esta capa se dividiría en tres componentes principales:

3.6.1. Agregaciones dinámicas de comportamiento

El primer componente (**`03_gold_customer_aggregations.py`**) se encarga de calcular el comportamiento histórico dinámico de las entidades (por ejemplo, cuánto ha gastado un cliente en la última hora, día o semana).

Para garantizar la exactitud temporal (*point-in-time correctness*) en el entrenamiento del modelo y no descartar ninguna transacción por desajustes temporales, la estrategia de diseño óptima abandona el enfoque de *streaming* con ventanas fijas y utiliza un **procesamiento por lotes (*batch*) con ventanas deslizantes (*rolling windows*)**. El proceso consiste en:

1. Leer el histórico completo de transacciones desde la capa plata.
2. Definir ventanas deslizantes (1 hora, 24 horas, 7 días, 30 días) ancladas al milisegundo exacto de cada transacción utilizando particiones por cliente (**`Window.partitionBy`**) e indicando el rango de tiempo hacia atrás.
3. Calcular todas las métricas específicas (conteos, sumas, medias, o recuento de anomalías) en una sola pasada.

Esta tabla finalizada permite calcular métricas derivadas complejas de un solo vistazo, como por ejemplo la ratio entre el gasto de las últimas 24 horas y la media de los últimos 30 días, conservando el contexto histórico exacto e individual de cada transacción.

3.6.2. Perfiles estáticos

El segundo componente (**`03_gold_customer_profile.py`**) procesa los datos dimensionales (el histórico creado en la capa plata) para derivar características estáticas o demográficas (por ejemplo, agrupar edades en categorías o clasificar niveles de ingresos).

Para evitar problemas de propagación en las actualizaciones del flujo de datos (como la pérdida silenciosa de la fecha de fin de validez **`__END_AT`** al usar flujos de solo inserción en *streaming*), este componente abandona el enfoque incremental. En su lugar, se diseña como una **vista materializada** (**`@dp.materialized_view`**) que realiza una lectura *batch* sobre la capa plata. De esta forma, el clúster es forzado a observar la foto histórica completa (*full refresh*) en cada ciclo del *pipeline*, garantizando que las fechas de validez del SCD2 se trasladen intactas. Al ser una tabla físicamente materializada, nos permite habilitar el *Change Data Feed (CDF)*, requisito indispensable para que el *online feature store* pueda sincronizar los cambios de los clientes eficientemente.

Integración nativa con el *feature store* Tanto la tabla física de agregaciones como la vista de perfiles están diseñadas para ser consumidas por el *feature store* de **`Databricks`**. Para que **`Unity Catalog`** reconozca automáticamente estos almacenes de características (sin necesidad de escribir código de registro adicional), se debe definir explícitamente su esquema (**`schema`**) incluyendo elementos críticos:

- **Restricción de clave primaria y serie temporal**: Se debe añadir una cláusula **`CONSTRAINT`** al final del esquema que defina la clave primaria (por ejemplo, **`customer_id`**) y marque la columna de tiempo (el **`timestamp`** de la transacción o el **`__START_AT`** del perfil) con la palabra clave **`TIMESERIES`**. Esto garantiza que, durante el entrenamiento, el *feature store* sepa exactamente qué versión histórica del cliente debe recuperar para evitar *point-in-time correctness*.
- ***CDF***: Para las tablas físicas, al definir las propiedades (**`table_properties`**), se debe habilitar **`"delta.enableChangeDataFeed": "true"`**. Esto permite que el almacén de características detecte y sincronice únicamente los registros que han cambiado (inserciones, actualizaciones o borrados) leyendo el *log* de transacciones de **`Delta`**, reduciendo drásticamente la latencia y los costes de computación al publicar los datos hacia el entorno *online*.

Para verificar que el registro automático ha funcionado, podéis dirigiros a la pestaña `Features` en el menú lateral de la plataforma, donde veréis vuestras tablas listas para ser servidas.

3.6.3. Tabla base o ancla

El tercer componente (**`03_gold_fraud_spine.py`**) genera la tabla *spine* (columna vertebral). Esta tabla es la base fundamental que se utilizará para el entrenamiento del modelo. Contiene exclusivamente los identificadores primarios (por ejemplo, **`customer_id`**), las marcas de tiempo exactas del evento (**`timestamp`**), la

variable objetivo a predecir y las características en tiempo real que llegan inherentemente con la petición (por ejemplo, importe, país del comercio y tipo de dispositivo). La naturaleza de la variable objetivo dependerá del tipo de problema: en un problema de clasificación será una etiqueta categórica (por ejemplo, **`is_fraud`**), mientras que en un problema de regresión será una variable continua (por ejemplo, el importe estimado de pérdida). Debéis adaptarla a vuestro caso de uso específico.

Es de vital importancia entender que en este *script* **no se unen** los datos del cliente ni las agregaciones calculadas en los pasos anteriores. La inyección de los perfiles y las agregaciones a la tabla *spine* se realizará de manera automática más adelante (en la fase de modelado) mediante las capacidades de cruce del *feature store*.

3.7. Orquestación de extremo a extremo y publicación de características

Para que el sistema de detección de fraude funcione de manera autónoma en producción, la lógica de transformación (capas bronce, plata y oro) y la puesta a disposición de las características deben integrarse en un único flujo de ejecución automatizado. Para ello, utilizamos los *Jobs* de **`Databricks`** (también conocidos como *Workflows*), que actúan como el motor central que orquesta y dispara (*triggers*) cada pieza de la infraestructura en el orden cronológico correcto.

Como se evidencia en la arquitectura desplegada para este proyecto, hemos creado un *Job* automatizado (denominado **`Credit Card Fraud Feature Pipeline`**) navegando a través de `Jobs & pipelines` → `Create` → `Job`. Al configurar un disparador (*Trigger*) programado para este *Job* (por ejemplo, con cadencia horaria), garantizamos que el ciclo completo se realice de forma totalmente desatendida. Este flujo de trabajo encadena secuencialmente dos tareas fundamentales:

3.7.1. Ejecución del *pipeline Medallion* y modos de infraestructura

La primera tarea del flujo (**`Run_Medallion_Pipeline`**) es de tipo *Pipeline* y dispara la ejecución de nuestro código orquestado por DLT. Se encarga de ingerir los nuevos datos crudos de la *landing zone*, aplicar las reglas de calidad en la capa plata y recalcular las agregaciones en la capa oro.

Dado que esta tarea está gobernada por el disparador programado del *Job*, el *pipeline* se ejecuta internamente en modo *Triggered* (por lotes o eventos disponibles). La gran ventaja arquitectónica de utilizar DLT es que **no tendríais que cambiar ni una sola línea de vuestro código en `Python`** para llevar este sistema a producción en tiempo real continuo. Si el caso de uso exigiera latencia cero y el presupuesto lo permitiera, bastaría con ir a los ajustes del *pipeline* y cambiar el `Pipeline mode` a `Continuous`. Vuestro código pasaría a procesar los datos en *streaming* ininterrumpido.

Nota importante para la memoria del proyecto: En este punto deberéis justificar vuestra capa de modelado de datos. Concretamente:

- Listar y explicar el significado analítico de las características temporales y estáticas que habéis decidido crear a partir de vuestros datos crudos.
- Argumentar por qué consideráis que dichas características aportarán valor predictivo al modelo de aprendizaje automático que desarrollaréis en la siguiente fase del proyecto.

3.7.2. Publicación en el *online feature store* para inferencia

Una vez que la capa oro ha sido actualizada por la primera tarea, la segunda tarea (**`Publish_to_Online_Store`**) entra en acción. Se trata de una ejecución de tipo *Notebook* que depende estrictamente de que el *pipeline* anterior finalice con éxito.

Aunque las tablas **`Delta`** de la capa oro son excelentes para el procesamiento analítico y el entrenamiento de modelos fuera de línea, sus tiempos de lectura no son adecuados para la inferencia en vivo. Para resolver esto, esta tarea ejecuta un *script* de configuración que registra y publica las tablas en un *online feature store* respaldado por **`Lakebase`**. El *online feature store* es una instancia gestionada que sirve características con latencia inferior a 10 milisegundos.

Es crucial destacar la estrategia de publicación implementada en esta fase:

- **Selección de tablas**: Se publican **`gold_customer_profile`** y **`gold_customer_aggregations`**, reconocidas automáticamente como tablas de características gracias a sus claves primarias. De este modo, la capa de *serving* puede recuperar el perfil demográfico y las señales de comportamiento actuales del cliente en tiempo real. Por el contrario, la tabla **`gold_fraud_spine`** no se publica aquí porque no es una tabla de características: es el andamio que el *Job* de entrenamiento usa para buscar las características.
- **Modo de sincronización**: En consonancia con la naturaleza del *Job* programado y la utilización de ventanas deslizantes en las agregaciones, el *script* utiliza el modo de publicación **`TRIGGERED`**. Este modo requiere que el CDF esté habilitado en las tablas de origen, lo que permite propagar exactamente los cambios acumulados desde la última sincronización y liberar todos los recursos de cómputo entre ejecuciones.

3.8. Consideraciones finales

El desarrollo de este apartado de la memoria debe dar respuesta a los diferentes puntos y requerimientos que se han ido señalando explícitamente a lo largo de esta guía. No obstante, como equipo, tenéis total libertad para ampliar el alcance, profundizar en la justificación de vuestras decisiones de diseño o incorporar detalles arquitectónicos adicionales si la naturaleza y complejidad de vuestro problema de negocio lo requieren.

En cuanto al formato, la extensión total recomendada para esta sección (cubriendo la infraestructura, ingesta en la capa bronce, reglas de calidad en la capa plata y creación de características en la capa oro) oscila entre las **5 y 10 páginas**.

Dentro de este límite, se valorará muy positivamente la capacidad de síntesis, el rigor técnico en las decisiones de diseño y el uso estratégico de elementos visuales de apoyo. Os animamos a enriquecer el documento incluyendo diagramas conceptuales de vuestra arquitectura de datos, capturas progresivas del DAG a medida que el *pipeline* crece fase a fase, o tablas resumen para ilustrar vuestros esquemas y reglas de validación.

4. Modelado y experimentación

Esta sección describe la tercera fase del ciclo de vida del proyecto. A diferencia de las fases anteriores, cuyo código reside en el *pipeline* de DLT, los cuadernos de esta fase se ejecutan de forma independiente o como tareas de un trabajo de **Databricks** y deben ubicarse en una carpeta dedicada dentro del repositorio del proyecto (por ejemplo, **notebooks** en el caso del proyecto de detección de fraude), separada del directorio **src** que contiene el código del *pipeline Medallion*. Todo el proceso está instrumentado con **MLflow** y gobernado por **Unity Catalog**, garantizando la trazabilidad de cada decisión y la reproducibilidad de cualquier resultado.

4.1. Cuadernos y *scripts* de la fase de modelado

La fase de modelado se estructura en siete cuadernos y *scripts* especializados con responsabilidades estrictamente separadas:

- **05_Training_Dataset_Generation.ipynb**: cuaderno de generación del conjunto de datos de entrenamiento. Combina la tabla *spine* con las tablas de características mediante el *point-in-time (PiT) join* del *feature store*, aplica las comprobaciones de calidad, filtra las instancias sin etiqueta y persiste el resultado como tabla **Delta** estática en **Unity Catalog** con sus metadatos de trazabilidad.
- **07_Utils.py**: utilidades compartidas entre todos los cuadernos de la fase. Define la configuración del proyecto, la partición temporal de los datos, la clasificación de columnas por tipo, la configuración del *pipeline* de preprocesado y las funciones de evaluación y visualización.
- **07_Training_Job.ipynb**: cuaderno de entrenamiento aislado. Recibe los hiperparámetros como parámetros de *widget*, construye y ajusta el *pipeline* completo y devuelve la ruta del modelo serializado y los ejemplos de entrada y salida.
- **07_Evaluation_Job.ipynb**: cuaderno de evaluación aislado. Carga el modelo desde su identificador uniforme de recursos (URI) de **MLflow**, lo evalúa sobre el conjunto indicado y devuelve las métricas y los artefactos de diagnóstico.
- **07_MLflow_Experimentation**: cuaderno orquestador del *grid search*. Lanza **07_Training_Job.ipynb** y **07_Evaluation_Job.ipynb** en sesiones aisladas de **Spark Connect** para cada punto del espacio de hiperparámetros, registra los resultados en **MLflow** y selecciona y registra el mejor candidato en **Unity Catalog**.
- **08_Utils.py**: utilidades compartidas exclusivas del ciclo de producción. Encapsula la lógica de lectura de metadatos del candidato desde **Unity Catalog**, la construcción del diccionario de hiperparámetros para el reentrenamiento, la gestión de aliases, la decisión de promoción, el registro de métricas y artefactos en **MLflow** y la limpieza de artefactos temporales.
- **08_Production.ipynb**: cuaderno del ciclo de producción. Implementa el patrón *champion-challenger* para evaluar el candidato sobre el conjunto de prueba y decidir su promoción a *champion*.

El aislamiento de entrenamiento y evaluación en cuadernos separados lanzados mediante **dbutils.notebook.run()** garantiza que cada punto del *grid* y cada evaluación en producción se ejecutan en una sesión de **Spark Connect** completamente limpia, eliminando el error de caché que afecta a la deserialización de modelos **Spark ML** en entornos *serverless*.

4.2. Generación del conjunto de datos de entrenamiento

El primer paso de esta fase es materializar el conjunto de datos de entrenamiento a partir de las tablas de la capa oro construidas en la fase anterior. Este paso se ejecuta una única vez por ciclo de reentrenamiento y su resultado se persiste como tabla **Delta** estática en **Unity Catalog** (por ejemplo,

gold_fraud_training_dataset en el caso del proyecto de detección de fraude), garantizando que todos los experimentos posteriores lean exactamente los mismos datos sin recalcular el cruce.

4.2.1. Cruce PiT con el *feature store*

El cruce se realiza mediante la interfaz de programación de aplicaciones (API) del *feature store* de **Databricks** usando **fe.create_training_set**. La tabla de partida es la tabla *spine* de la capa oro (por ejemplo, **gold_fraud_spine**), que actúa como esqueleto del conjunto de datos: define qué filas lo componen, el instante temporal de cada transacción y la etiqueta de supervisión. Sobre ella se declaran los objetos **FeatureLookup** necesarios, uno por cada tabla de características de la capa oro (por ejemplo, **gold_customer_profile** para el perfil estático del cliente y **gold_customer_aggregations** para las métricas comportamentales con ventana deslizante).

El parámetro **timestamp_lookup_key** activa el *join* temporal **AS OF**: para cada transacción de la *spine*, el sistema recupera la versión de las características del cliente que era válida en el instante exacto de esa transacción. Esto elimina por diseño cualquier fuga de datos del futuro (*data leakage*), garantizando que el modelo solo aprende de información que habría estado disponible en producción en el momento de la predicción.

Se debe excluir explícitamente del conjunto de datos resultante aquellas columnas que son metadatos operativos de auditoría y que no estarían disponibles en una petición real al *endpoint* de inferencia (por ejemplo, **label_available_date**).

4.2.2. Comprobaciones de calidad y resultados

Antes de persistir el conjunto de datos, se realizan tres verificaciones sobre el **DataFrame** materializado:

- **Consistencia del volumen**: el número de filas del conjunto enriquecido debe coincidir exactamente con el de la *spine* original, verificando que el PiT *join* no ha introducido duplicidades ni ha descartado transacciones.
- **Nulos en características**: se contabilizan los valores faltantes por columna. En las características estáticas, los nulos son esperables cuando el PiT *join* viaja a fechas anteriores al primer registro de perfil del cliente. En las características de ventana, los nulos corresponden a entidades sin historial en esa ventana temporal. Ambos escenarios se delegan al *pipeline* de modelado mediante imputación.
- **Balance de clases**: se analiza la distribución de la variable objetivo. Las instancias con etiqueta nula corresponden a eventos recientes aún no auditados y deben descartarse antes de persistir el conjunto de datos, ya que un algoritmo supervisado no puede aprender de instancias sin etiqueta.

Nota importante para la memoria del proyecto: Debéis documentar aquí los resultados concretos de estas tres comprobaciones: el recuento total de filas, el número de nulos por característica y la distribución de clases con sus porcentajes. El fuerte desequilibrio entre clases, si existe, justifica la estrategia de pesos que se aplica en el clasificador.

Caso de uso: fraude en tarjetas de crédito

El conjunto de datos generado contiene 60.000.000 filas, coincidiendo exactamente con la *spine*. El balance de clases presenta 58.108.076 transacciones legítimas (96.85 %), 1.757.224 fraudes confirmados (2.93 %) y 134.700 transacciones con etiqueta nula (0.22 %), correspondientes a operaciones recientes aún no auditadas que se descartan antes de persistir. El fuerte desequilibrio confirma la necesidad de aplicar pesos de clase en el clasificador.

4.2.3. Persistencia y trazabilidad

El conjunto de datos se guarda en modo **`overwrite`**, pero **`Delta Lake`** conserva automáticamente todas las versiones anteriores mediante *time travel*. Para garantizar la trazabilidad entre datos y modelo, se persisten cuatro metadatos como propiedades de la tabla: la versión semántica (**`ml.delta_semantic_version`**), que la libreta de experimentación registrará en **`MLflow`** como parámetro del experimento; la versión física (**`ml.delta_physical_version`**), que apunta al *snapshot* exacto del **`WRITE`** para el *time travel* desde la libreta de entrenamiento; la fecha máxima de los datos (**`ml.data_max_date`**); y la fecha máxima del ciclo anterior (**`ml.data_previous_max_date`**), que define el inicio de la ventana de prueba en el siguiente reentrenamiento.

4.3. Arquitectura del *pipeline*

El preprocesado y el clasificador se encadenan en un único objeto **`Pipeline`** de **`Spark MLlib`**. Al empaquetar todas las etapas de transformación junto al modelo, se garantiza que las mismas estadísticas aprendidas durante el entrenamiento se aplican durante la inferencia, eliminando el *training-serving skew*. Debéis documentar y justificar en vuestra memoria las etapas concretas de vuestro *pipeline* y las decisiones de diseño tomadas en cada una.

Caso de uso: fraude en tarjetas de crédito

El *pipeline* consta de ocho etapas secuenciales: imputación de valores nulos con la mediana; conversión de indicadores booleanos de seguridad a **`DOUBLE`**; ingeniería de cinco características derivadas computables en tiempo real (método de alto riesgo, protocolo de internet (IP) extranjera, dispositivo no reconocido, logaritmo del importe y operación transfronteriza en canal *online*); indexación categórica por frecuencia descendente; codificación *one-hot* eliminando la última categoría para evitar multicolinealidad; ensamblaje en un único vector de características; selección por varianza; y escalado estándar a media cero y desviación estándar uno. Los identificadores de alta cardinalidad (**`customer_id`**, **`transaction_id`**, **`merchant_id`**, **`mcc_code`**) y la columna temporal (**`timestamp`**) se excluyen del vector de características antes de la clasificación de columnas, de forma que permanecen disponibles en el **`DataFrame`** tras el **`transform()`**, lo que permite su uso para trazabilidad y como claves de *join* en producción. Para mitigar el fuerte desbalance de clases, el clasificador **`LogisticRegression`** aplica pesos calculados dinámicamente mediante frecuencia inversa, produciendo pesos aproximados de 20.51 para la clase fraude y 0.51 para la clase legítima.

4.4. Optimización de hiperparámetros

La selección del modelo se aborda mediante un *grid search* sobre el espacio de hiperparámetros del clasificador, utilizando una partición temporal fija que respeta la causalidad de los datos y evita la fuga de información del futuro hacia el pasado. Debéis documentar la estrategia de partición adoptada y justificar los rangos de hiperparámetros explorados en relación con la naturaleza de vuestro problema.

La métrica de selección debe ser la más adecuada para vuestro problema. En escenarios de clasificación con fuerte desbalance de clases, el área bajo la curva de precisión-exhaustividad (AUC-PR) es generalmente preferible a la exactitud o al área bajo la curva *ROC* (AUC-ROC) porque penaliza explícitamente los falsos positivos en el contexto de la clase minoritaria.

Para las métricas de clasificación, se realiza un barrido de umbral de decisión y se registra el que maximiza el *F1-score* en validación como **`best_threshold_val`**, que se propaga a todos los reentrenamientos posteriores. Por cada configuración se registran en **`MLflow`** los artefactos de diagnóstico: curva precisión-exhaustividad (PR), curva característica operativa del receptor (ROC), matriz de confusión, curva de calibración, barrido de umbral y coeficientes del modelo.

Caso de uso: fraude en tarjetas de crédito

La partición temporal adoptada es la siguiente: entrenamiento desde 2019-12-31 hasta 2022-12-31, validación desde 2023-01-01 hasta 2023-12-31, y prueba desde 2024-01-01 hasta 2024-12-31. El *grid* explora el parámetro de regularización (**`reg_param`**) y la mezcla elástica (**`elastic_net_param`**). La siguiente tabla resume los resultados de las seis configuraciones evaluadas.

`reg_param`	`elastic_net`	AUC-PR validación	AUC-ROC validación	F1 validación (umbral óptimo)	Umbral
0.001	0.0	0.746	0.943	0.707	0.92
0.001	0.5	0.745	0.943	0.707	0.92
0.01	0.0	0.741	0.943	0.704	0.91
0.01	0.5	0.738	0.943	0.702	0.89
0.1	0.0	0.717	0.939	0.681	0.82
0.1	0.5	0.667	0.918	0.655	0.73

La configuración ganadora corresponde a **`reg_param`** = 0.001 y **`elastic_net_param`** = 0.0, obteniendo un AUC-PR de validación de 0.746 y un umbral óptimo de 0.92. Este modelo se registra en **`Unity Catalog`** como versión 1 bajo el alias **`candidate`**.

4.5. Evaluación en producción y ciclo *champion-challenger*

Una vez seleccionado el candidato, la fase de evaluación en producción implementa el patrón estándar de la industria para despliegues seguros: el candidato compite como ***challenger*** contra el modelo actualmente en producción (***champion***) y solo se promueve si demuestra un rendimiento estrictamente superior sobre el conjunto de prueba, que permanece reservado hasta este momento para garantizar su integridad como estimador imparcial del rendimiento real.

El flujo de aliases en **`Unity Catalog`** que documenta el estado de cada versión es el siguiente:

Alias	Significado	¿Quién lo asigna?
`candidate`	Mejor modelo del *grid search*, pendiente de evaluación	Experimentación
`challenger`	Candidato en evaluación activa sobre el conjunto de prueba	Producción
`champion`	Modelo reentrenado sobre el histórico completo y en producción	Producción
`retired`	Antiguo *champion* superado por el nuevo	Producción
`rejected`	*Challenger* que no superó al *champion*	Producción

El *challenger* se reentrena sobre entrenamiento y validación combinados antes de ser evaluado en prueba, garantizando que el modelo que compite ha aprendido de todos los datos históricos disponibles antes del corte temporal. Si el *challenger* gana, se realiza un reentrenamiento final sobre el histórico completo, ya que la evaluación ha cumplido su función y el conjunto de prueba deja de estar reservado. El modelo registrado como *champion* es este reentrenamiento final. La tabla *baseline* se construye a partir de las características crudas del conjunto de prueba, sin aplicar las transformaciones internas del *pipeline*, más la predicción y la probabilidad de fraude del *champion*, garantizando que su esquema coincida con el de **`gold_fraud_inference_enriched`**, la tabla que **`Databricks Lakehouse Monitoring`** monitoriza en producción. Toda la ejecución queda registrada en una única ejecución de **`MLflow`** con las métricas de ambos modelos, el *delta* de AUC-PR y la decisión tomada.

Nota importante para la memoria del proyecto: Debéis documentar los resultados de la evaluación en prueba del *challenger* y, si existe, del *champion*, indicando la decisión de promoción y su justificación. Si es el primer ciclo, indicad que no existe *champion* previo y que el *challenger* se promueve directamente.

Caso de uso: fraude en tarjetas de crédito

En el primer ciclo no existe *champion* previo (arranque en frío), por lo que el *challenger* se promueve directamente. Los resultados sobre el conjunto de prueba son: AUC-PR = 0.757, AUC-ROC = 0.944, F1 = 0.980. El modelo se reentrena sobre el histórico completo y se registra como versión 2 en **`Unity Catalog`** bajo el alias **`champion`**. El conjunto de prueba en su forma original, enriquecido con la predicción y la probabilidad de fraude generadas por el *champion*, se persiste en la tabla **`gold_fraud_test_baseline`**. Esta tabla actúa como referente del monitor de **`Databricks Lakehouse Monitoring`** para el cálculo de *data drift* de características, métricas de rendimiento y métricas de equidad en producción.

4.6. Orquestación del ciclo de reentrenamiento

Para automatizar el ciclo completo, se crea en **`Databricks`** un trabajo compuesto por tres tareas secuenciales (por ejemplo, denominado **`Credit Card Fraud Retraining Pipeline`** en el caso de referencia):

1. **`Generate_Training_Dataset`**: genera el conjunto de datos de entrenamiento mediante el PiT *join* con el *feature store* y lo persiste en **`Unity Catalog`**.
2. **`Run_Hyperparameter_Search`**: ejecuta el *grid search* completo, registra los resultados en **`MLflow`** y promueve el mejor modelo al alias **`candidate`** en **`Unity Catalog`**.
3. **`Evaluate_And_Promote`**: ejecuta el ciclo *champion-challenger* sobre el conjunto de prueba y, si corresponde, registra el nuevo *champion*.

El trabajo se activa manualmente para el primer ciclo de entrenamiento. En producción, el disparador automático será una alerta sobre las tablas de métricas de **`Databricks Lakehouse Monitoring`**: cuando la métrica de selección real en producción descienda por debajo del umbral definido respecto al *baseline* del *champion*, la alerta dispara el trabajo automáticamente, cerrando el ciclo de operaciones de aprendizaje automático (MLOps) de forma completamente desatendida.

4.7. Consideraciones finales

El desarrollo de este apartado de la memoria debe evidenciar la conexión entre las decisiones técnicas y los objetivos de negocio definidos en la fase de alcance y viabilidad. En particular, debéis justificar la elección del algoritmo en relación con el tipo de problema, la selección de la métrica primaria y su alineación con los KPIs económicos del proyecto, y la estrategia de partición temporal adoptada.

En cuanto al formato, la extensión total recomendada para esta sección oscila entre las **4 y 8 páginas**. Se valorará positivamente la inclusión de capturas de la interfaz de **`MLflow`** que ilustren la comparativa de experimentos, tablas resumen con los resultados del *grid search* y los diagramas del flujo de aliases en **`Unity Catalog`**.